Rapport de Stage d'Été

Automatisation de la Documentation des Jobs Talend sur Confluence

Conception et mise en œuvre de pipelines n8n pilotés par IA pour la synchronisation automatique de la documentation technique ETL

Réalisé par :
Ahmed MESSOUDI

Encadrante :
Mme. Hela SMAOUI

Stage d'été effectué au sein de la société **OliveSoft**

Résumé

Dans le cadre de mon stage d'été au sein de la société **OliveSoft**, sous l'encadrement de Mme. Hela SMAOUI, j'ai conçu et implémenté deux pipelines d'automatisation destinés à résoudre un problème récurrent en ingénierie des données : l'obsolescence de la documentation technique des jobs ETL Talend. Ces pipelines, bâtis sur la plateforme d'orchestration **n8n**, s'appuient sur une architecture événementielle reliant **GitHub**, la **Talend Management Console (TMC)**, un modèle de langage (**LLM** hébergé via Groq) et **Atlassian Confluence**.

Le premier workflow, déclenché automatiquement à chaque *push* sur le dépôt GitHub, implémente une logique de « Version Gatekeeper » qui ne déclenche la régénération de la documentation que lorsqu'une nouvelle version du job a effectivement été déployée en production, évitant ainsi tout bruit inutile dans Confluence. Le second workflow, conçu pour des besoins de migration et de retraitement par lot, permet de régénérer l'ensemble de la documentation d'un dossier Confluence source vers un dossier cible, en s'appuyant sur une correspondance automatique entre les pages Confluence et les fichiers source présents sur GitHub.

Les deux systèmes partagent un socle commun : extraction structurelle des fichiers `.item` (XML Talend), génération automatique de diagrammes de séquence via intelligence artificielle, publication idempotente (création ou mise à jour) dans Confluence, et traçabilité des versions synchronisées via une table de suivi. Ce rapport présente la problématique, l'architecture technique retenue, le détail fonctionnel des deux workflows ainsi que les défis techniques rencontrés et résolus au cours du stage.

Mots-clés : Automatisation, ETL, Talend, n8n, Confluence, Intelligence Artificielle, LLM, Intégration Continue, Documentation Technique.

Remerciements

Je tiens à exprimer ma sincère gratitude à l'ensemble de l'équipe d'**OliveSoft** pour son accueil chaleureux et les conditions de travail favorables mises à ma disposition tout au long de ce stage d'été.

Mes remerciements les plus vifs s'adressent à mon encadrante, **Mme. Hela SMAOUI**, pour sa disponibilité constante, ses conseils avisés et son accompagnement tout au long de ce projet. Sa confiance et sa pédagogie m'ont permis de mener à bien cette mission technique tout en développant mes compétences en automatisation, en intégration de systèmes et en intelligence artificielle appliquée.

Je remercie également l'ensemble des équipes techniques d'OliveSoft pour leur disponibilité et leurs retours constructifs, qui ont contribué à l'amélioration continue des workflows développés durant ce stage.

Table des matières

Résumé **1**

Remerciements **2**

1 Introduction **6**

2 Contexte et Problématique **7**
2.1 Le défi de la documentation technique en environnement ETL 7
2.1.1 Obsolescence de la documentation 7
2.1.2 Complexité de l'extraction des métadonnées 7
2.1.3 Incohérence des versions entre code source et production 7
2.1.4 Manque de visibilité sur le *Data Lineage* 8
2.2 Formulation de la problématique 8

3 Objectifs du Stage **9**
3.1 Objectif général 9
3.2 Objectifs spécifiques 9

4 Architecture Technique et Stack Technologique **10**
4.1 n8n – Orchestration et Automatisation 10
4.2 GitHub – Contrôle de Version et Source de Vérité 10
4.3 Talend Management Console (TMC) – Gestion des Déploiements 10
4.4 Atlassian Confluence – Base de Connaissances 11
4.5 Intelligence Artificielle et Visualisation – Groq API et Mermaid.js 11
4.6 n8n Data Tables – Gestion d'État et Versioning 11

5 Workflow 1 – Synchronisation Événementielle par Déclenchement GitHub **12**
5.1 Vue d'ensemble et objectif 12
5.2 Phase 1 – Déclenchement et Filtrage 12
5.3 Phase 2 – Contrôle de Version (*Version Gatekeeper*) 12
5.4 Phase 3 – Analyse Structurelle et Extraction (Parsing XML) 13
5.5 Phase 4 – Génération de Diagrammes par Intelligence Artificielle 13
5.6 Phase 5 – Génération HTML et Synchronisation Confluence 13
5.7 Diagramme de Séquence du Workflow 1 14

6 Workflow 2 – Régénération par Lot depuis un Dossier Confluence **15**
6.1 Contexte et cas d'usage 15
6.2 Phase 1 – Découverte des Jobs 15

6.3 Phase 2 – Appariement avec le Dépôt GitHub 15
6.4 Phase 3 – Extraction et Génération IA 16
6.5 Phase 4 – Publication par Lot (Logique *Upsert*) 16
6.6 Diagramme de Séquence du Workflow 2 16

7 Défis Techniques et Solutions Apportées 18
7.1 Troncature des réponses du modèle de langage 18
7.2 Fiabilité du traçage des éléments (*Paired Item*) 18
7.3 Gestion des conflits de titre lors de la création de pages 18
7.4 Limitations des intégrations LangChain natives 19
7.5 Optimisation et consolidation de la logique métier 19

8 Bénéfices et Valeur Ajoutée 20
8.1 Réduction de la dette technique 20
8.2 Source unique de vérité (*Single Source of Truth*) 20
8.3 Amélioration de la visibilité sur le flux de données 20
8.4 Optimisation des ressources et du bruit documentaire 20
8.5 Flexibilité opérationnelle 21

9 Conclusion et Perspectives 22

Table des figures

5.1 Diagramme de séquence du Workflow 1 – Synchronisation événementielle par push GitHub 14

6.1 Diagramme de séquence du Workflow 2 – Régénération par lot depuis un dossier Confluence 17

Chapitre 1

Introduction

Dans le paysage actuel de l'ingénierie des données, la maintenance d'une documentation technique à jour constitue un défi majeur et souvent sous-estimé. Les projets ETL (*Extract, Transform, Load*), et en particulier ceux développés sous **Talend**, génèrent une complexité structurelle importante : règles de mapping entre champs sources et cibles, requêtes SQL et SOQL embarquées, appels vers des API externes (REST, SOAP, Salesforce), et enchâinements de sous-jobs orchestrant des flux de données critiques pour l'entreprise.

Historiquement, la rédaction de ces spécifications techniques repose sur un processus manuel : un ingénieur ETL doit, après chaque modification significative d'un job, retranscrire à la main les changements dans une page Confluence dédiée. Ce processus, en plus d'être chronophage, est structurellement fragile : il dépend entièrement de la discipline individuelle des développeurs et n'est, dans la pratique, que rarement respecté avec la rigueur nécessaire.

C'est dans ce contexte que s'inscrit le présent stage d'été, réalisé au sein de la société **OliveSoft**. L'objectif confié a été de concevoir une solution d'automatisation complète, capable de générer, mettre à jour et synchroniser automatiquement la documentation technique des jobs Talend, en s'appuyant sur une architecture événementielle moderne et sur l'intelligence artificielle générative pour produire des représentations visuelles (diagrammes de séquence) compréhensibles par des équipes aussi bien techniques que métier.

Ce rapport présente en détail la démarche suivie : l'analyse de la problématique initiale, les choix d'architecture technique, l'implémentation de deux workflows complémentaires répondant à des cas d'usage distincts (synchronisation continue à chaque déploiement, et régénération par lot pour des besoins de migration), ainsi que les difficultés techniques rencontrées et les solutions apportées.

Chapitre 2

Contexte et Problématique

2.1 Le défi de la documentation technique en environnement ETL

Le développement et la maintenance de jobs ETL soulèvent plusieurs problématiques critiques que ce projet a vocation à adresser directement.

2.1.1 Obsolescence de la documentation

La documentation rédigée manuellement est fréquemment négligée par les développeurs, faute de temps ou par simple oubli, ou mise à jour avec un retard significatif par rapport aux évolutions réelles du code. Il en résulte un *drift* (décalage) croissant entre le comportement effectif du job en production et la description qui en est faite dans la base de connaissances. Ce décalage constitue un risque opérationnel : toute personne se fiant à une documentation périmée pour intervenir sur un job peut introduire des régressions ou mal interpréter un flux de données.

2.1.2 Complexité de l'extraction des métadonnées

Les fichiers sources produits par Talend (extension `.item`) sont en réalité des fichiers XML volumineux et complexes, encodant à la fois la structure graphique du job (composants, connexions) et sa logique métier (expressions Java, requêtes SQL, paramétrage de contexte). Extraire manuellement les règles de mapping (composant `tMap`), les requêtes SQL/SOQL, les paramètres de contexte et les flux de données représente une tâche fastidieuse, répétitive et particulièrement sujette à l'erreur humaine.

2.1.3 Incohérence des versions entre code source et production

Une modification du code dans le dépôt Git ne signifie pas nécessairement qu'un déploiement a eu lieu en production. Si la synchronisation de la documentation était déclenchée à chaque *commit*, cela générerait un bruit considérable et de multiples versions inutiles dans Confluence, correspondant à des états intermédiaires du code jamais réellement déployés. Il est donc nécessaire de disposer d'un mécanisme de contrôle capable de distinguer une simple modification de code d'un déploiement effectif.

2.1.4 Manque de visibilité sur le *Data Lineage*

Comprendre le flux de données (*data pipeline*) d'un job de bout en bout nécessite généralement une analyse approfondie et manuelle du graphe Talend, une information souvent absente ou insuffisamment détaillée dans les documentations textuelles classiques. Cette absence de représentation visuelle claire ralentit considérablement la prise en main d'un job par une nouvelle personne ou l'analyse d'impact lors d'une évolution.

2.2 Formulation de la problématique

L'ensemble de ces constats peut se résumer par la question centrale suivante :

> *« Comment garantir, sans effort manuel récurrent, que la documentation technique des jobs Talend disponible dans Confluence reflète en permanence l'état réellement déployé en production, tout en offrant une représentation visuelle exploitable du flux de données ? »*

C'est autour de cette problématique que s'est articulé l'ensemble du travail réalisé durant ce stage.

Chapitre 3

Objectifs du Stage

3.1 Objectif général

L'objectif général confié pour ce stage était de concevoir, développer et mettre en production une solution d'automatisation « *Zero-Touch* » (sans intervention manuelle) assurant la génération et la synchronisation continue de la documentation technique des jobs ETL Talend vers Confluence.

3.2 Objectifs spécifiques

Pour répondre à cet objectif général, plusieurs objectifs spécifiques ont été définis conjointement avec l'encadrante du stage :

- Mettre en place un pipeline événementiel se déclenchant automatiquement lors d'un dépôt de code (*push* GitHub), avec un filtrage strict ne considérant que les fichiers Talend pertinents.
- Implémenter un mécanisme de contrôle de version (« *Version Gatekeeper* ») interrogeant la Talend Management Console (TMC) afin de ne déclencher la documentation que pour des jobs réellement déployés.
- Développer un moteur d'extraction structurelle capable d'analyser le format XML natif de Talend et d'en extraire les métadonnées fonctionnelles pertinentes (sources, cibles, requêtes, mappings, paramètres).
- Intégrer un modèle de langage (LLM) afin de générer automatiquement, à partir de la structure abstraite du job, un diagramme de séquence au format **Mermaid.js** illustrant le flux de données.
- Automatiser la publication (création ou mise à jour) des pages de spécification dans Confluence, avec gestion cohérente du versionnage.
- Concevoir un second workflow complémentaire, orienté traitement par lot, permettant de régénérer massivement la documentation d'un ensemble de jobs à partir d'un dossier Confluence source vers un dossier cible, notamment pour des besoins de migration ou de refonte de l'espace documentaire.
- Assurer la robustesse et la fiabilité des deux pipelines face aux aléas propres à l'utilisation de modèles d'intelligence artificielle en production (troncature de réponses, incohérences de format, erreurs de correspondance de données).

Chapitre 4

Architecture Technique et Stack Technologique

Pour répondre aux enjeux décrits précédemment, une architecture logicielle moderne, orientée événementielle et basée sur des services *cloud* interconnectés via API a été retenue. Cette section présente chacun des composants de la stack technologique ainsi que leur rôle respectif dans le système global.

4.1 n8n – Orchestration et Automatisation

n8n constitue le moteur central du système. Il s'agit d'une plateforme d'automatisation de workflows à bas code (*low-code*), permettant de modéliser visuellement des enchaînements d'appels API, de transformations de données et de logiques conditionnelles sous forme de graphes de nœuds. Dans ce projet, n8n orchestre l'intégralité du cycle de vie de la documentation : déclenchement, extraction, appel au modèle d'intelligence artificielle, et publication finale. Son modèle d'exécution basé sur des nœuds de type Code (JavaScript) a également permis d'implémenter une logique métier fine là où les nœuds standards ne suffisaient pas (analyse XML, comparaison de versions sémantiques, extraction robuste de contenu généré par IA).

4.2 GitHub – Contrôle de Version et Source de Vérité

Le dépôt **GitHub** héberge l'ensemble des fichiers `.item` constituant le code source des jobs Talend. Il fait office de source de vérité (*source of truth*) pour le contenu technique des jobs. Les *webhooks* GitHub sont utilisés comme déclencheurs (*triggers*) du premier workflow, tandis que l'API REST de contenu de dépôt (*Git Trees API*) est utilisée dans le second workflow pour retrouver, par appariement de noms, le fichier source correspondant à une page Confluence donnée.

4.3 Talend Management Console (TMC) – Gestion des Déploiements

L'API de la **Talend Management Console** est utilisée comme « *Gatekeeper* » (portier) du système : avant tout traitement coûteux (extraction, appel IA, publication), le

workflow interroge la TMC afin de vérifier qu'une nouvelle version du job a effectivement été déployée. Cette étape est essentielle pour éviter de documenter des modifications de code qui ne sont jamais allées en production.

4.4 Atlassian Confluence – Base de Connaissances

Confluence constitue la cible finale de la documentation générée. Son API REST (v1 et v2 selon les besoins) est utilisée pour la création de nouvelles pages, la mise à jour de pages existantes (avec incrémentation de version), ainsi que pour l'interrogation de la structure hiérarchique des espaces (dossiers, pages enfants) nécessaire au second workflow.

4.5 Intelligence Artificielle et Visualisation – Groq API et Mermaid.js

L'intelligence artificielle intervient à une étape précise et ciblée du pipeline : la génération du diagramme de séquence représentant le flux de données du job. La structure abstraite du job (participants et échanges identifiés lors du parsing XML) est transmise à un modèle de langage hébergé via l'API **Groq**, choisie pour sa latence d'inférence très faible, un critère important dans un pipeline devant traiter potentiellement plusieurs dizaines de jobs. Le modèle génère en sortie un diagramme au format **Mermaid.js**, un langage textuel de description de diagrammes, qui est ensuite converti en image (via le service `mermaid.ink`) pour être intégré directement dans la page Confluence publiée.

4.6 n8n Data Tables – Gestion d'État et Versioning

Enfin, les **Data Tables** natives de n8n sont utilisées comme mécanisme de persistance léger, permettant de stocker, pour chaque job, la dernière version documentée ainsi que l'identifiant de la page Confluence associée. Cette table de suivi joue un double rôle : éviter les traitements redondants (comparaison de versions) et permettre une logique de publication idempotente (choix entre création et mise à jour d'une page).

TABLE 4.1 – Synthèse de la stack technologique

Composant	**Rôle dans l'architecture**
n8n	Orchestration centrale, logique métier, transformation de données
GitHub	Source de vérité du code, déclenchement événementiel, récupération des fichiers `.item`
Talend Management Console (TMC)	Contrôle des versions réellement déployées en production
Groq API (LLM)	Génération automatique des diagrammes de séquence
Mermaid.js	Représentation textuelle et rendu visuel des diagrammes
Atlassian Confluence	Publication et hébergement final de la documentation
n8n Data Tables	Suivi des versions synchronisées et des identifiants de pages

Chapitre 5

Workflow 1 – Synchronisation Événementielle par Déclenchement GitHub

5.1 Vue d'ensemble et objectif

Le premier workflow conçu durant ce stage implémente une logique de synchronisation *continue* : chaque fois qu'un job Talend est modifié et poussé sur le dépôt GitHub, puis effectivement déployé en production via la Talend Management Console, la documentation correspondante dans Confluence est automatiquement régénérée et publiée. Ce workflow constitue le cœur opérationnel du système, conçu pour fonctionner de manière autonome et répétée, sans aucune intervention humaine.

Il est structuré en cinq phases distinctes et séquentielles, détaillées ci-après.

5.2 Phase 1 – Déclenchement et Filtrage

Le workflow est initié par un événement de type *push* sur le dépôt GitHub, capté via un nœud `GitHub Trigger`. Un premier filtre strict est ensuite appliqué : seuls les fichiers possédant l'extension `.item` et situés dans le répertoire `process/` du dépôt sont pris en compte. Ce filtrage garantit que des modifications non pertinentes (fichiers de configuration, documentation annexe, ressources statiques) n'interfèrent pas avec le déclenchement du pipeline et n'entraînent pas de traitements inutiles.

5.3 Phase 2 – Contrôle de Version (*Version Gatekeeper*)

Cette phase constitue le cœur de la logique métier du workflow. Avant tout traitement du fichier modifié, le système interroge l'API de la Talend Management Console afin d'obtenir la version actuellement **déployée** du job concerné (`tmcVersion`). Cette version est ensuite comparée à la dernière version documentée, stockée dans la Data Table de suivi de n8n (`dbVersion`).

— **Si** `tmcVersion > dbVersion` : le job a bien été déployé depuis la dernière documentation générée ; le workflow poursuit son exécution.

— **Si les deux versions sont identiques** : aucune action n'est nécessaire, le workflow s'arrête immédiatement pour ce job, évitant tout appel API ou traitement IA superflu.

Cette logique de « portier de version » constitue l'un des apports les plus significatifs du système : elle découple explicitement la notion de *modification de code* de celle de *déploiement effectif*, garantissant que Confluence ne reçoit jamais de mise à jour correspondant à un état intermédiaire ou non déployé du code.

5.4 Phase 3 – Analyse Structurelle et Extraction (Parsing XML)

Une fois le déploiement confirmé, le fichier `.item` concerné est téléchargé depuis GitHub. Un moteur d'extraction déterministe, implémenté en JavaScript au sein d'un nœud Code n8n, analyse la structure XML du fichier afin d'en extraire :

— les métadonnées générales du job (auteur, date de dernière modification, objectif) ;

— les sources et cibles de données (fichiers plats, bases de données, appels API) ;

— les requêtes SQL et SOQL embarquées dans les composants d'entrée/sortie ;

— les règles de transformation portées par les composants de mapping (`tMap`) ;

— les paramètres de contexte (*job parameters*) nécessaires à l'exécution du job.

Ce parsing déterministe (sans intervention de l'IA à ce stade) garantit l'exactitude et la fiabilité des données factuelles extraites, l'intelligence artificielle n'intervenant que dans une étape ultérieure, purement dédiée à la synthèse visuelle du flux.

5.5 Phase 4 – Génération de Diagrammes par Intelligence Artificielle

La structure abstraite du job (liste des nœuds fonctionnels et de leurs connexions, débarrassée des composants purement techniques ou utilitaires tels que la journalisation ou la gestion des connexions) est transmise à un modèle de langage via l'API Groq. Le modèle génère en réponse un diagramme de séquence au format **Mermaid.js**, illustrant visuellement le pipeline de données du job : sources sollicitées, étapes de transformation, et cibles alimentées.

5.6 Phase 5 – Génération HTML et Synchronisation Confluence

Les données extraites lors de la phase 3 et le diagramme généré lors de la phase 4 sont compilés en un document HTML structuré et stylisé, conforme au format de spécification d'intégration attendu. Le système interroge ensuite l'API Confluence pour déterminer si une page existe déjà pour ce job :

— si la page existe, elle est **mise à jour** (requête `PUT`) avec incrémentation du numéro de version Confluence ;

— si la page n'existe pas encore, elle est **créée** (requête `POST`).

Enfin, la Data Table de suivi de n8n est mise à jour afin de refléter la nouvelle version désormais synchronisée, clôturant ainsi le cycle pour ce job.

5.7 Diagramme de Séquence du Workflow 1

La figure 5.1 présente le diagramme de séquence complet du workflow, illustrant les interactions entre le développeur, GitHub, l'orchestrateur n8n, la Talend Management Console, la table de versions, le modèle Groq AI LLM et Confluence, à travers les quatre phases décrites ci-dessus (regroupées visuellement en : déclenchement/contrôle de version, extraction/génération IA, publication intelligente, et livraison).

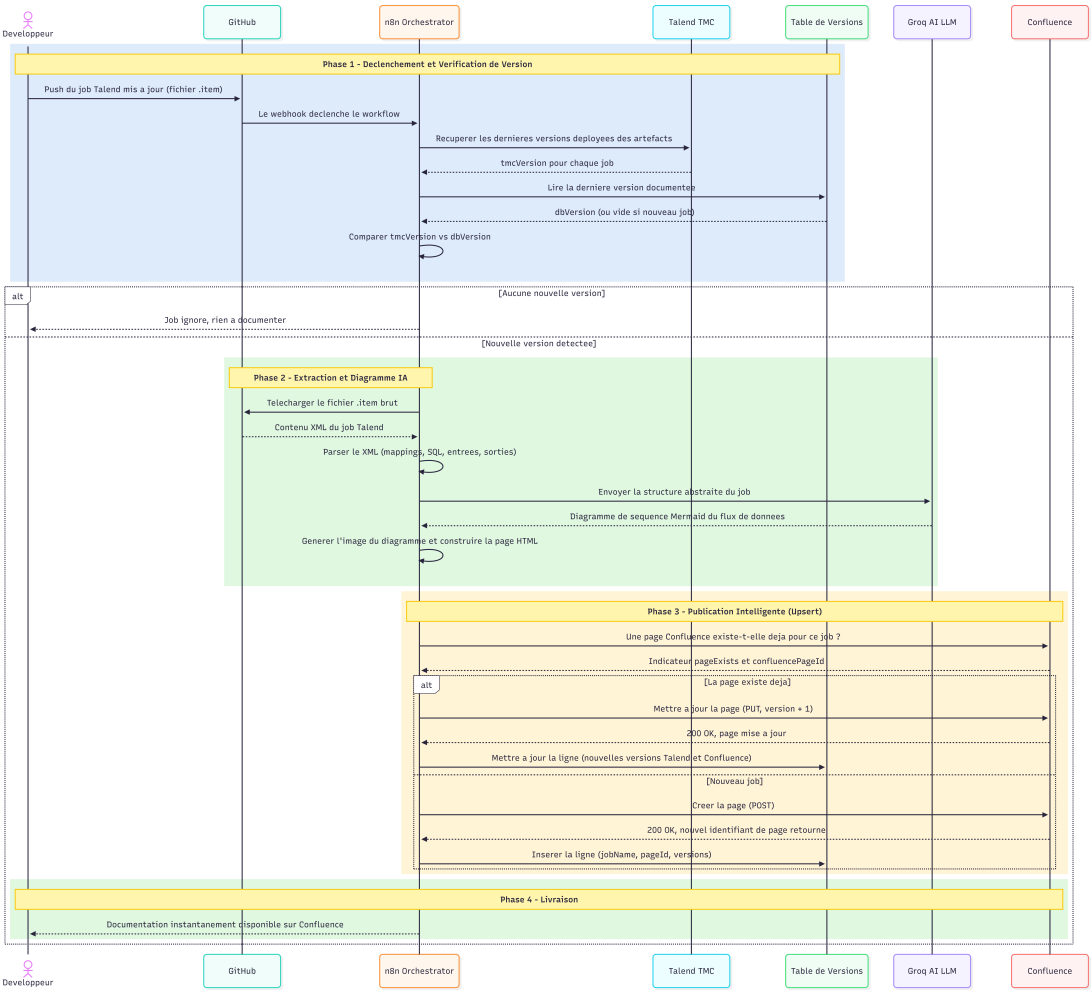


FIGURE 5.1 – Diagramme de séquence du Workflow 1 – Synchronisation événementielle par push GitHub

Chapitre 6

Workflow 2 – Régénération par Lot depuis un Dossier Confluence

6.1 Contexte et cas d'usage

Si le premier workflow répond à un besoin de synchronisation continue, un second besoin est apparu au cours du stage : celui de pouvoir **régénérer massivement** la documentation d'un ensemble de jobs, indépendamment de tout événement de déploiement individuel. Ce besoin se présente typiquement dans trois situations : la migration d'un espace Confluence existant vers une nouvelle structure documentaire, la refonte du gabarit de spécification appliqué à l'ensemble des jobs déjà documentés, ou encore le traitement initial (*backfill*) d'un ensemble de jobs jamais documentés automatiquement.

Pour répondre à ce besoin, un second workflow a été conçu, fonctionnant par lot à partir d'un **dossier Confluence source** et publiant vers un **dossier Confluence cible** distinct, garantissant ainsi qu'aucune page existante n'est modifiée de manière destructive pendant la phase de migration.

6.2 Phase 1 – Découverte des Jobs

Contrairement au premier workflow, celui-ci est déclenché manuellement (ou pourrait être placé sur une planification périodique). Il débute par l'interrogation de l'API Confluence afin de lister l'ensemble des pages contenues dans le dossier source configuré. Les titres de ces pages sont ensuite analysés afin d'en extraire les noms de jobs correspondants, selon la convention de nommage établie (suffixe « – Integration Specification »).

6.3 Phase 2 – Appariement avec le Dépôt GitHub

Chaque nom de job identifié est ensuite mis en correspondance avec l'arborescence complète du dépôt GitHub (récupérée via l'API *Git Trees*), afin de retrouver le fichier `.item` correspondant, situé dans le répertoire `process/`. Lorsque plusieurs versions d'un même fichier sont présentes, seule la version la plus récente (selon un numéro de version sémantique extrait du nom de fichier) est retenue. Les jobs pour lesquels aucun fichier correspondant n'est trouvé sont journalisés séparément afin de faciliter leur traitement manuel ultérieur.

6.4 Phase 3 – Extraction et Génération IA

Une fois le fichier source identifié, les phases d'extraction structurelle (parsing XML) et de génération du diagramme de séquence par intelligence artificielle sont strictement identiques, sur le plan fonctionnel, à celles du premier workflow. La génération du diagramme repose ici sur un nœud `Basic LLM Chain` du framework LangChain, connecté à un modèle Groq, offrant une intégration native avec l'écosystème n8n pour la gestion des invites (*prompts*) et des réponses du modèle.

6.5 Phase 4 – Publication par Lot (Logique *Upsert*)

Pour chaque job traité, le système interroge la table de suivi afin de déterminer si une page correspondante existe déjà dans le **dossier cible**. Selon le résultat, la page est soit mise à jour (`PUT`), soit créée (`POST`) directement sous le dossier cible grâce au paramètre `ancestors` de l'API Confluence, garantissant que l'intégralité de la documentation régénérée reste organisée dans l'arborescence prévue à cet effet.

6.6 Diagramme de Séquence du Workflow 2

Le diagramme de séquence du second workflow, dont le code source Mermaid est fourni en Annexe A, suit la même structuration en phases que le premier workflow (découverte, extraction, génération IA, publication, livraison), tout en reflétant les spécificités du traitement par lot : bouclage sur l'ensemble des jobs du dossier source, appariement explicite avec GitHub, et publication ciblée vers un dossier distinct.

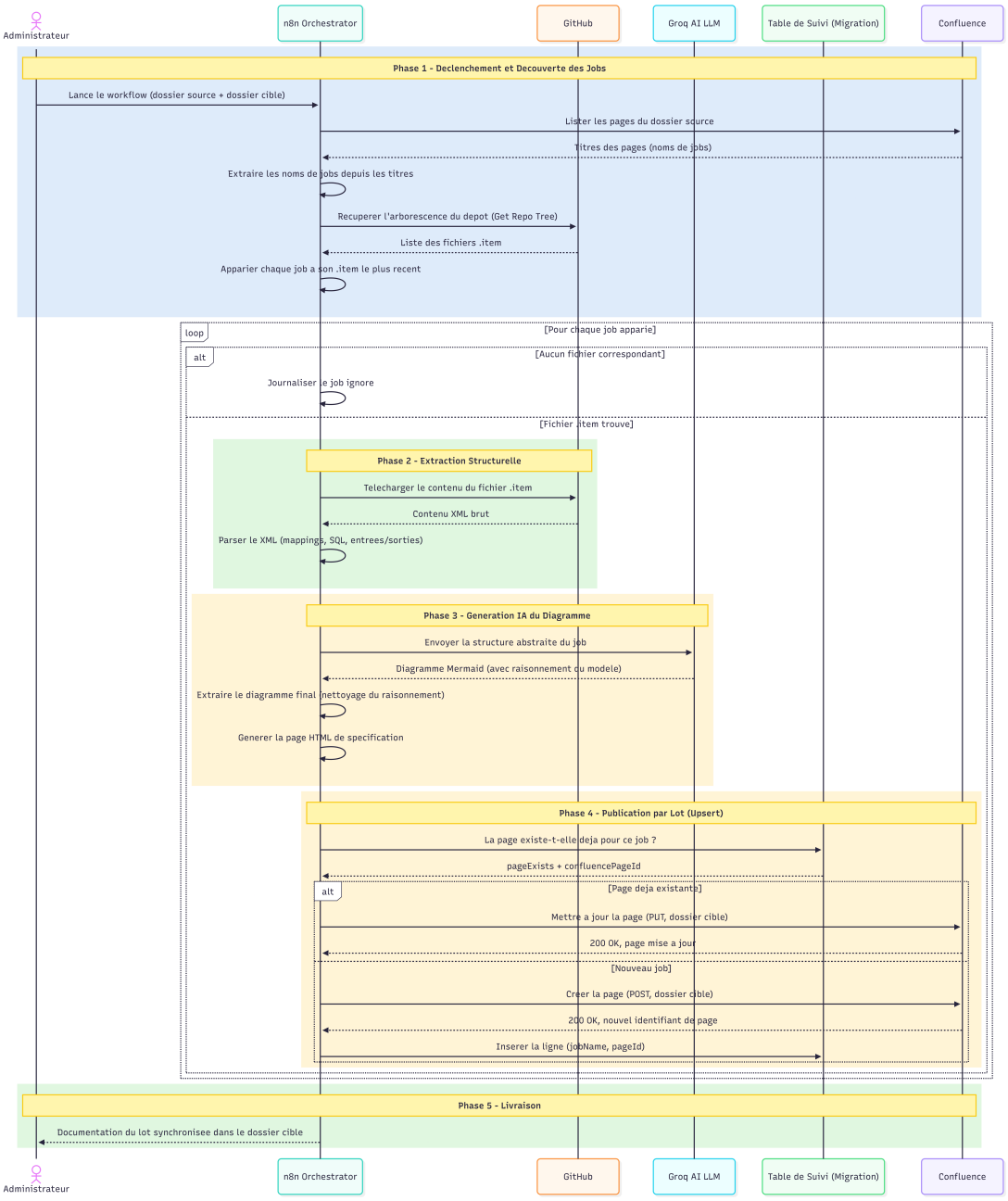


FIGURE 6.1 – Diagramme de séquence du Workflow 2 – Régénération par lot depuis un dossier Confluence

Chapitre 7

Défis Techniques et Solutions Apportées

L'intégration d'un modèle de langage au sein d'un pipeline de production a soulevé plusieurs difficultés techniques non triviales, dont la résolution a occupé une part importante du travail réalisé durant ce stage.

7.1 Troncature des réponses du modèle de langage

Certains modèles utilisés (notamment les modèles dits « de raisonnement ») produisent, avant leur réponse finale, un bloc de raisonnement interne délimité par des balises `<think>...</think>`. Lorsque le budget de tokens alloué à la génération est insuffisant, la réponse peut être interrompue *pendant* ce raisonnement, avant que le diagramme final ne soit produit. Une première version de l'extracteur, se contentant de rechercher la première occurrence du mot-clé `sequenceDiagram` dans le texte brut, était alors susceptible de récupérer un fragment de brouillon issu du raisonnement plutôt que le diagramme final. La solution retenue consiste à détecter explicitement l'absence de balise fermante : dans ce cas, la génération est considérée comme échouée et signalée comme telle, plutôt que de tenter une extraction incertaine.

7.2 Fiabilité du traçage des éléments (*Paired Item*)

Le moteur d'exécution de n8n s'appuie sur un mécanisme de « traçage d'éléments appariés » (*paired item*) permettant, à tout moment, de retrouver l'élément d'entrée ayant produit un élément de sortie donné. Il a été constaté que ce traçage n'est pas toujours préservé par le nœud `Basic LLM Chain` lorsqu'il traite plusieurs invites au sein d'une même exécution par lot, provoquant une erreur bloquante lors de l'utilisation de la fonction `itemMatching()`. La solution retenue consiste à envelopper cet appel dans un bloc `try/catch`, avec un repli automatique vers une correspondance positionnelle simple, garantissant la continuité du traitement même lorsque le traçage natif est indisponible.

7.3 Gestion des conflits de titre lors de la création de pages

Lors de la mise en production du second workflow, un cas particulier a été identifié : si la table de suivi interne n'est pas parfaitement synchronisée avec l'état réel de Confluence,

le système peut tenter de créer une page dont le titre existe déjà, provoquant une erreur `400` de l'API Confluence. Cette situation a mis en évidence l'importance de traiter la table de suivi comme une source secondaire d'information, et non comme une source de vérité absolue quant à l'existence d'une page.

7.4 Limitations des intégrations LangChain natives

Il a également été constaté que certains paramètres avancés proposés par les API des fournisseurs de modèles (tel que le paramètre `reasoning_effort` permettant de désactiver le raisonnement interne d'un modèle) ne sont pas systématiquement exposés par les nœuds d'intégration LangChain de n8n. Ce constat a orienté certains choix d'implémentation vers des appels HTTP directs à l'API du fournisseur, offrant un contrôle plus fin sur les paramètres de génération, au prix d'une intégration moins native avec l'écosystème n8n.

7.5 Optimisation et consolidation de la logique métier

Au fil du développement, plusieurs nœuds de type Code exécutant des étapes logiquement liées (par exemple, l'extraction des noms de jobs depuis les titres Confluence, suivie immédiatement de leur appariement avec les fichiers GitHub) ont été fusionnés en un unique nœud. Cette consolidation réduit le nombre d'étapes d'exécution du workflow, simplifie sa lecture visuelle, et limite la surface d'erreurs liée à la transmission de données entre nœuds successifs.

Chapitre 8

Bénéfices et Valeur Ajoutée

La mise en œuvre de ces deux workflows apporte des avantages tangibles à l'organisation, aussi bien sur le plan de la qualité documentaire que sur celui de l'efficacité opérationnelle.

8.1 Réduction de la dette technique

L'automatisation complète du cycle de documentation élimine quasi-totalement le temps auparavant consacré à la rédaction et à la mise à jour manuelle des spécifications techniques, libérant un temps d'ingénierie significatif.

8.2 Source unique de vérité (*Single Source of Truth*)

Grâce à la logique de « Version Gatekeeper », la documentation publiée dans Confluence est en permanence alignée avec la version effectivement déployée en production sur la Talend Management Console, éliminant tout risque de décalage entre le réel et le documenté.

8.3 Amélioration de la visibilité sur le flux de données

La génération automatique de diagrammes de séquence offre aux équipes, aussi bien techniques que métier, une visibilité immédiate et intuitive sur les flux de données, réduisant considérablement le temps nécessaire à la prise en main d'un job existant.

8.4 Optimisation des ressources et du bruit documentaire

La logique de contrôle de version évite les appels API inutiles ainsi que la création de versions « fantômes » dans Confluence, correspondant à des états de code jamais réellement déployés.

8.5 Flexibilité opérationnelle

La combinaison des deux workflows – l'un événementiel et continu, l'autre manuel et orienté traitement par lot – offre à l'équipe une flexibilité opérationnelle complète : synchronisation automatique au quotidien, et capacité de régénération massive en cas de migration ou de refonte documentaire.

Chapitre 9

Conclusion et Perspectives

Ce stage a permis de démontrer comment l'orchestration de workflows via n8n, couplée à une analyse structurelle rigoureuse des fichiers source et à l'utilisation ciblée de l'intelligence artificielle générative, peut transformer en profondeur la gouvernance de la documentation technique au sein d'une organisation manipulant des pipelines ETL complexes.

En automatisant intégralement la documentation des jobs Talend, la solution mise en œuvre garantit non seulement une qualité documentaire supérieure et constamment à jour, mais libère également le temps des ingénieurs de données, qui peuvent désormais se concentrer sur des tâches à plus forte valeur ajoutée.

Sur le plan personnel, ce stage a été l'occasion de développer des compétences variées : conception d'architectures événementielles, intégration d'API multiples (GitHub, Talend TMC, Confluence, Groq), traitement et analyse de fichiers XML complexes, et surtout, expérience concrète de l'intégration d'un modèle de langage au sein d'un système de production, avec l'ensemble des contraintes de fiabilité et de robustesse que cela implique.

Plusieurs pistes d'amélioration pourraient être envisagées pour prolonger ce travail : la mise en place de tests automatisés sur les diagrammes générés afin d'en vérifier syntaxiquement la validité avant publication, l'ajout d'un mécanisme de nouvelle tentative (*retry*) automatique en cas de troncature de la réponse du modèle de langage, ou encore l'extension du système de contrôle de version à d'autres types de jobs et d'environnements de déploiement au sein de l'organisation.

Bibliographie

[1] n8n Documentation officielle, https://docs.n8n.io, consulté en 2026.

[2] Talend Management Console – Documentation API, Qlik/Talend, consulté en 2026.

[3] Atlassian Confluence REST API Documentation, https://developer.atlassian.com/cloud/confluence/, consulté en 2026.

[4] Mermaid.js Documentation, https://mermaid.js.org, consulté en 2026.

[5] Groq API Documentation, https://console.groq.com/docs, consulté en 2026.

[6] GitHub REST API Documentation, https://docs.github.com/en/rest, consulté en 2026.